

UNIDAD 5. PROGRAMACIÓN MODULAR: FUNCIONES, MÉTODOS Y PROCEDIMIENTOS

**PROGRAMACIÓN
CFGS DAW**

Adaptado y ampliado por:
José Miguel Blázquez

Autores originales:
Carlos Cacho
Raquel Torres
Lionel Tarazon
Fco. Javier Valero

2023/2024

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

 Importante

 Atención

 Interesante

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original. NOTA: Esta es una obra derivada de la obra original realizada por Carlos Cacho y Raquel Torres.

ÍNDICE DE CONTENIDOS

1. Introducción a la programación modular.....	4
1.1 Funciones, métodos y procedimientos.....	4
2. Funciones.....	6
2.1 Componentes de una función. Declaración.....	6
2.2 Ejemplos de funciones.....	7
2.3 Llamar o invocar funciones.....	9
2.4 Parámetros de una función: paso por valor y paso por referencia.....	12
2.5 Devolución de valores.....	13
3. Ámbito de las variables.....	15
4. Diseño de funciones.....	16
5. Diseño descendente y modularidad.....	17
5.1 Independencia funcional: acoplamiento y cohesión.....	17
5.2 Ventajas de un buen diseño descendente.....	18
6. Recursividad.....	19
7. Librerías y API.....	20

UD05. PROGRAMACIÓN MODULAR: FUNCIONES Y MÉTODOS

1. INTRODUCCIÓN A LA PROGRAMACIÓN MODULAR

Hasta ahora, todos los programas que hemos hecho constaban de un único bloque principal dentro de "main". Sin embargo, a medida que un programa crece y se vuelve más complejo, este bloque aumenta en extensión y puede hacerse cada vez más difícil de gestionar. La mejor forma de crear y mantener un programa grande es construirlo a partir de piezas más pequeñas o módulos; cada uno de los cuales es más manejable que el programa en su totalidad.

Esto, en desarrollo de software, alimenta el “diseño descendente”, que se basa en resolver un problema general descomponiéndolo en programas más simples, denominados subproblemas. Se trata del famoso enfoque para resolución de problemas llamado “Divide y vencerás”.

✈ Las **funciones** (subprogramas) son utilizadas para **evitar la repetición de código** al poder ejecutarlo desde varios puntos de un programa con sólo invocarlo.

El concepto de función es una forma de **encapsular** un conjunto de instrucciones dentro de una declaración específica (llamada generalmente SUBPROGRAMA o FUNCIÓN), permitiendo la descomposición funcional y la diferenciación de tareas, en lo que se denomina **programación modular**, lo cual supone una evolución de la programación **estructurada**, y el paso previo a la programación **orientada a objetos**.

1.1 Funciones, métodos y procedimientos

En Java, a las funciones se les llama **métodos**, cuando veamos programación orientada a objetos estudiaremos el porqué. De esta forma, las funciones son realmente un caso particular de método, existiendo otros tipos especiales. Es muy común entre programadores que se hable indistintamente de estos 3 términos, y aunque representan el mismo concepto, poseen algunas pequeñas diferencias importantes. Veamos una aproximación interesante a estas diferencias, independientes del lenguaje:

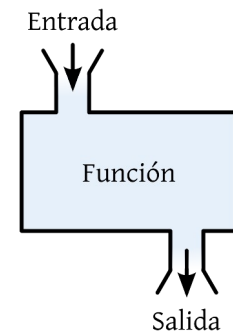
Funciones:

Las funciones son un conjunto de líneas de código (instrucciones) encapsuladas en un bloque. Suelen recibir parámetros, cuyos valores utilizan para efectuar operaciones, y que devuelven un valor al terminar. En otras palabras, una función puede recibir parámetros o argumentos (algunas no reciben nada), hace uso de dichos valores recibidos como sea necesario y devuelve un valor usando la instrucción **return**; si no devuelve nada entonces no es una función. En java las funciones usan el modificador **static** al declararlas, por lo que se consideran técnicamente métodos estáticos. Podríamos considerarlas similares al concepto de cualquier función matemática.

Métodos:

Los métodos y las funciones en Java están en capacidad de realizar las mismas tareas, es decir, son funcionalmente idénticos, pero su diferencia radica en la manera en que hacemos uso de uno u otro (el contexto). Un método también puede recibir valores, efectuar operaciones con estos y devolver un resultado, sin embargo, el método está asociado a una clase u objeto, SIEMPRE. Básicamente podemos decir que un método es una función que pertenece a un objeto o clase, mientras que una función existe por sí sola, sin necesidad de un objeto para ser usada; por lo tanto, el método es un concepto inherente a la programación orientada a objetos.

* **Nota:** recordad que, en Java, al ser todo objetos, las funciones como tal no existen, se les sigue llamando métodos. No es técnicamente correcto emplear el vocablo “funciones” en Java, aunque de manera informal es válido si nos resulta útil para diferenciar el concepto. También es válido entre aquellas personas que están aprendiendo Java pero aún no han llegado a la orientación a objetos.



Procedimientos:

Los procedimientos son básicamente un conjunto de instrucciones que se ejecutan sin devolver ningún valor, hay quienes dicen que un procedimiento no recibe valores o argumentos, sin embargo en la definición no hay nada que se lo impida. En el contexto de Java un procedimiento es básicamente un método cuyo tipo de retorno es *void*, por lo que no nos obliga a utilizar una sentencia **return**. Este “procedimiento” de ejemplo no devuelve nada, se limita a realizar una acción:

```
public static void procedimiento(int n, String nombre){ //Fíjate en el void
    if(n > 0 && !nombre.equals("")) //usamos los dos parámetros
        System.out.println("hola " + nombre);
}
```

¡No devuelve nada!

🔊 **Ya has usado funciones anteriormente** sin ser consciente. Por ejemplo, cuando invocas funciones matemáticas de la clase “Math” lo estás haciendo. Recuerda:

`Math.sqrt(9);` → Devuelve la raíz de 9, es decir, 3. Esa función calcula la raíz cuadrada cuando la invocas pasándole como parámetro el valor a calcular.

También lo has hecho con todas las funciones de tratamiento de cadenas de la clase String (contains, indexOf, replace, fill), etcétera...

2. FUNCIONES

Como hemos comentado al principio de la unidad, en programación, una función es un bloque de código que realiza una tarea específica y puede ser llamado desde otras partes del programa. La idea central es encapsular una tarea o un conjunto de tareas relacionadas en una unidad independiente y reutilizable.

Utilidades principales de las funciones:

- Agrupar código que forma una entidad propia o una idea concreta.
- Agrupar código que se necesitará varias veces en un programa, con la misión de no repetir dicho código cada vez. Este ahorro tiene múltiples ventajas, como facilitar el mantenimiento.
- Dividir el código de un programa grande en subprogramas (funciones), cada uno de ellos especializados en resolver una parte del problema, mejorando la estructuración del mismo.

Características de las funciones:

- Se definen mediante un **nombre único** que representa el bloque de código.
- Pueden ser **llamadas** (ejecutadas) desde cualquier parte del código.
- Se les puede **pasar valores** para que los procesen de alguna forma.
- Pueden **devolver** un resultado para ser usado desde donde se les llamó.

2.1 Componentes de una función. Declaración.

Declarar una función simplemente significa crearla para que luego pueda ser llamada (utilizada) desde otro lugar del código de nuestro programa. Una función se estructura en **cabecera** y **cuerpo**. Veamos los componentes clave:

- La **cabecera** se declara en una sola línea y se compone de:
 - **Modificadores**: existen muchos pero los veremos en futuras unidades, especialmente en orientación a objetos. Por ahora solo utilizaremos public static.
 - **Tipo devuelto**: el tipo de dato que devolverá la función, como por ejemplo int, double, char, boolean, String, etc. Si la función no devuelve nada se indica mediante void.
 - **Nombre de la función**: identificador único para llamar a la función.
 - **Lista de parámetros ó argumentos**: indica los tipos y nombres de los datos que se le pasarán a la función cuando sea llamada. Pueden ser uno, varios o ninguno. Habitualmente no más de 4 ó 5. Si hay varios se separan por comas.
- El **cuerpo** es un bloque de código entre llaves { ... } que contiene el código que se ejecutará cuando desde otra parte del código llamemos a la función.

La estructura general es la siguiente:

```
[modificadoresAcceso] tipoDevuelto nombreFunción([parámetros]) {  
    // instrucciones  
    [return valor;]  
}
```

Veamos un ejemplo simple y real de dicha estructura:

```
public static int sumar(int num1, int num2) {  
    int resultado = num1 + num2;  
    return resultado;  
}
```

- **Modificadores:** public static. Significan que la función puede ser utilizada desde cualquier parte del programa (public) y que la función puede ser usada sin POO (static).
- **Tipo devuelto:** int
- **Nombre de la función:** sumar
- **Lista de parámetros:** num1 (int) y num2 (int).
- **Instrucciones:** creación de la variable entera “resultado” y almacenamiento de la suma de los 2 parámetros recibidos.
- **Return:** esta función sí tiene return y devuelve el valor de la variable creada anteriormente, es decir, el “resultado” de la suma de num1 y num2.

Veamos otro **ejemplo**. En este caso la función “max” devuelve el máximo de entre 2 números que le pasamos como parámetros. Fíjate en su estructura:

```
public static int max(int x, int y) {  
    if(x > y)  
        return x;  
    else  
        return y;  
}
```

Modificadores Tipo devuelto Nombre Parámetros

Cuerpo de la función

2.2 Ejemplos de funciones

```
public static void imprimeHolaMundo() {  
    System.out.println("Hola mundo");  
}
```

Este es un ejemplo muy sencillo de una función llamada ‘imprimeHolaMundo’, que no tiene parámetros de entrada (no hay nada entre los paréntesis) y no devuelve ningún valor (indicado por void). Cuando la llamemos lo único que hará será escribir por pantalla el mensaje “Hola mundo”.

```
public static void imprimeHolaNombre(String nombre) {  
    System.out.println("Hola " + nombre);  
}
```

Esta función se llama ‘imprimeHolaNombre’, tiene como parámetro de entrada un dato String llamado ‘nombre’ y no devuelve nada. Cuando la llamemos imprimirá por pantalla el texto “Hola “ seguido del String nombre que se le pase como parámetro.

```
public static int doble(int a) {  
    int resultado = a * 2;  
    return resultado;  
}
```

Esta función se llama 'doble', tiene como parámetro de entrada un dato int llamado 'a' y devuelve un dato de tipo int. Cuando la llamemos calculará el doble de 'a' y lo devolverá (con el return).

```
public static int multiplica(int a, int b) {  
    int resultado = a * b;  
    return resultado;  
}
```

Esta función se llama 'multiplica', tiene dos parámetros de entrada de tipo int llamados 'a' y 'b' y devuelve un dato de tipo int. Cuando la llamemos calculará a*b y lo devolverá (con el return).

```
public static double maximo(double valor1, double valor2) {  
    double max;  
    if (valor1 > valor2)  
        max = valor1;  
    else  
        max = valor2;  
    return max;  
}
```

Esta función se llama 'maximo', tiene dos parámetros de entrada de tipo double llamados 'valor1' y 'valor2' y devuelve un dato de tipo double. Cuando la llamemos calculará el máximo entre 'valor1' y 'valor2' y lo devolverá.

```
public static int sumaVector(int[] v) {  
    int suma = 0;  
    for (int i = 0; i < v.length; i++)  
        suma = suma + v[i];  
    return suma;  
}
```

Esta función se llama 'sumaVector', tiene un parámetro de entrada tipo int[] (un vector de int) llamado 'v' y devuelve un dato tipo int. Cuando la llamemos recorrerá el vector 'v', calculará la suma de todos sus elementos y la devolverá.

Es importante saber que **las funciones se declaran dentro de 'class' pero fuera del 'main'**.

```
3 public class programadeprueba {
4
5     public static void imprimeHolaMundo() {
6         System.out.println("Hola mundo");
7     }
8
9     public static int doble(int a) {
10         int resultado = a * 2;
11         return resultado;
12     }
13
14     public static int multiplica(int a, int b) {
15         int resultado = a * b;
16         return resultado;
17     }
18
19     public static void main(String[] args) {
20
21         // Un programa siempre empieza ejecutándose por la función main.
22         // Aquí va el código principal de nuestro programa.
23     }
24
25 }
26
27
```

En este programa tenemos 4 funciones: **imprimeHolaMundo**, **doble**, **multiplica** y **main**. Sí, el bloque del **'main'** donde siempre has programando hasta ahora es en efecto una función en sí misma, pero un poco especial: **'main'** es la función principal, el punto de inicio de un programa. Es obligatorio que todo programa Java tenga una función **main**. Si te fijas, es una función que recibe como parámetro un `String[]` (vector de Strings) y no devuelve nada (aunque podría devolver un `int`). El porqué de esto lo veremos más adelante. **Las 3 funciones que hemos declarado** (declarar significa describir su funcionamiento) **arriba del main por sí solas no hacen nada, simplemente están ahí esperando a que sean llamadas** (utilizadas), normalmente desde el propio **main**. Veámoslo.

2.3 Llamar o invocar funciones

Las funciones pueden ser **invocadas o llamadas** desde cualquier otra función, incluida ella misma. Sí, una función puede llamar a cualquier otra función, y una función puede llamarse a sí misma. De todos modos, **por ahora llamaremos a funciones solo desde la función principal 'main'**. Así es más sencillo de aprender al principio.

Cuando se invoca una función, el **flujo de ejecución** salta a la función (pasándole los parámetros si los hubiera), se ejecutan las instrucciones de la función y por último, el flujo de ejecución vuelve al punto que llamó a la función para seguir ejecutándose desde allí.

Las funciones se invocan con su nombre, pasando la lista de parámetros entre paréntesis. Si no tiene parámetros han de ponerse los paréntesis igualmente, pero vacíos. Si la función devuelve un valor, para recogerlo hay que asignarlo a una variable o utilizarlo de algún modo (pueden combinarse funciones en expresiones e incluso pasarlo a otras funciones).

Ejemplo de “main” utilizando las funciones del apartado anterior:

```

public static void main(String[] args) {
    // No tiene parámetros ni devuelve valor. Muestra “Hola Mundo”
    imprimeHolaMundo();

    // Llamada a función y almacenamiento del resultado
    int a = doble(10); // a = 20    (10*2)
    int b = multiplica(3, 5); // b = 15    (3*5)

    // Pueden pasarse variables como parámetros
    int c = doble(a); // c = 40    (20*2)
    int d = multiplica(a, b); // d = 300    (20*15)

    // Pueden combinarse funciones y expresiones
    int e = doble(4) + multiplica(2,10); // e = 8 + 20

    System.out.println("El doble de 35 es: " + doble(35));
    // “El doble de 35 es 70”

    System.out.println("12 por 12 es " + multiplica(12,12));
    // “12 por 12 es 144”
}

```

Ejemplo completo 1: Programa con una función que suma dos números.

```

4   public class Suma {
5
6       public static void main(String[] args) {
7           Scanner sc = new Scanner(System.in);
8           int num1, num2, suma;
9
10          System.out.print("Introduce un número: ");
11          num1 = sc.nextInt();
12
13          System.out.print("Introduce otro número: ");
14          num2 = sc.nextInt();
15
16          suma = suma(num1, num2);
17
18          System.out.println("La suma es: " + suma);
19      }
20
21      public static int suma(int n1, int n2) {
22          int suma;
23
24          suma = n1 + n2;
25
26          return suma;
27      }
28
29  }
30
31  }

```

Salida:

```

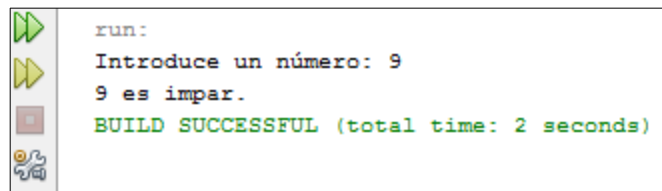
run:
Introduce un número: 3
Introduce otro número: 4
La suma es: 7
BUILD SUCCESSFUL (total time: 3 seconds)

```

Ejemplo completo 2: Programa con una función que determina si un número es par o impar.

```
14 public class ParImpar {
15
16     public static void main(String[] args) {
17         Scanner in = new Scanner(System.in);
18         int num;
19
20         System.out.print("Introduce un número: ");
21         num = in.nextInt();
22
23         if(par(num) == true) // Llamada a la función desde la expresión
24             System.out.println(num + " es par.");
25         else
26             System.out.println(num + " es impar.");
27     }
28
29     public static boolean par(int numero)
30     {
31         boolean par = false;
32
33         if(numero % 2 == 0) // Si el resto es 0 par será 'true' sino 'false'
34             par = true;
35
36         return par;
37     }
38 }
```

Salida:



```
run:
Introduce un número: 9
9 es impar.
BUILD SUCCESSFUL (total time: 2 seconds)
```

2.4 Parámetros de una función: paso por valor y paso por referencia

Existen dos tipos de parámetros y es importante comprender la diferencia.

- **Parámetros de tipo simple (paso por valor):** int, double, boolean, char, etc. En este caso **se pasan por valor**. Es decir, **el valor se copia al parámetro** y por lo tanto si se modifica dentro de la función esto no afectará al valor fuera de ella porque **son variables distintas**.

```
public static void main(String[] args) {
    int a = 10;
    System.out.println("Valor inicial de a: " + a);
    // a vale 10
    imprime_doble(a); // Se le pasa el 10 a la función
    System.out.println("Valor final de a: " + a); // a sigue valiendo 10
}

// El parámetro 'a' es independiente de la 'a' del main.

public static void imprime_doble(int a) {
    // Se copia el valor 10 a esta nueva 'a'
    a = 2 * a; // Duplica el valor de 'a' dentro de la función, no fuera
    System.out.println("Valor de a en la función: " + a);
    // 'a' vale 20 en este ámbito
}
```

SALIDA:

```
run:
Valor inicial de a: 10
Valor de a en la función: 20
Valor final de a: 10
```

- **Parámetros de tipo objeto (paso por copia de referencia):** con objetos de tipo String, los Arrays, etc. En este caso no se copia el contenido del objeto sino que **se le pasa a la función una COPIA de la referencia al objeto original (un puntero)**. Por ello **desde la función se accede directamente al objeto** que se encuentra fuera. **Los cambios que hagamos dentro de la función afectarán al objeto externo**. NO es aconsejable proceder así. Debemos evitar modificar variables externas a una función desde dentro de ella, aunque sean estructuras u objetos.

```
// Suma x a todos los elementos del vector v
public static void suma_x_vector(int v[], int x) {
    for (int i = 0; i < v.length; i++)
        v[i] = v[i] + x;
}

public static void main(String[] args) {
    int v[] = {0, 1, 2, 3};
    System.out.println("Vector antes: " + Arrays.toString(v));
    suma_x_vector(v, 10);
    System.out.println("Vector después: " + Arrays.toString(v));
}
```

SALIDA:

```
run:
Vector antes: [0, 1, 2, 3]
Vector después: [10, 11, 12, 13]
```

Aunque las variables de tipo objeto puedan ser modificadas dentro de la función, no debemos usar esta técnica para devolver valores fuera de la función a menos que no nos quede más remedio. Usaremos el `return` y el tipo adecuado en su lugar. Si necesitamos devolver “varios” valores, probablemente el diseño de la función no sea el adecuado. Hablaremos de eso en el apartado del diseño descendente.

IMPORTANTE: Como un parámetro de tipo objeto es una referencia al objeto `String` o `Array` que está fuera de ella, si se le asigna otro objeto se pierde la referencia y ya no se puede acceder al objeto fuera de la función. Aunque Java permite hacerlo, **no se aconseja hacerlo, es mala práctica.**

```
/* Asignamos a x un nuevo vector, por lo que x dejará de apuntar al
vector original. ¡El vector original no cambia! Simplemente ya no podemos
acceder a él desde x porque se ha perdido la referencia a dicho objeto.
*/

public static void funcion1(int x[]) {
    x = new int[10];
    // x apuntará a un nuevo vector, el original queda intacto
    // lo que hagamos aquí con x no afectará al vector original
}
```

NO SE ACONSEJA HACER LO ANTERIOR.

2.5 Devolución de valores

✈ Los métodos pueden devolver valores de tipo básico o primitivo (`int`, `double`, `boolean`, etc.) y también de tipo objeto (`Strings`, `arrays`, etc.).

En todos los casos es el comando **`return`** el que realiza esta labor. En el caso de `arrays` y `objetos`, devuelve una referencia a ese `array` u `objeto`. Es conveniente que una función devuelva un resultado mediante la sentencia `return`, por lo que es conveniente evitar las funciones “**`void`**” en la medida de lo posible.

Sintaxis: `return expresion;`

2 ejemplos equivalentes:

```
public static double cubo (double x) {
    return x*x*x;
}

-----

public static double cubo (double x) {
    int cubo = x*x*x;
    return cubo;
}
```

¡Hacen lo mismo! Fíjate en cómo se emplea la sentencia **`return`** en ambos casos.

En algunas ocasiones no es necesario que el método estático tenga que devolver un valor al finalizar su ejecución. En este caso, el tipo de dato que debe indicar en la cabecera de declaración del método es el tipo **void** y la sentencia **return** debe omitirse. Si no hay sentencia **return** dentro de un método, su ejecución continúa hasta que se alcanza el final del método y entonces se devuelve la secuencia de ejecución al lugar dónde se invocó al método. Un método cuyo tipo de retorno no es **void** necesita siempre devolver algo. Si el código de un método contiene varias sentencias if, hay que asegurarse de que cada una de las posibles opciones devuelve un valor. En caso contrario, se generaría un error de compilación. Por ejemplo:

```
public static boolean esPositivo(int x) {  
    if (x < 0) return false;  
    if (x > 0) return true;  
    // Error: retorno perdido si x es igual a cero porque no está contemplado  
}  
}
```

Ejemplo de intento de compilación del código anterior:

```
$>javac PruebaPositivo.java  
pruebaPositivo.java:14: missing return statement  
    }  
    ^
```

3. ÁMBITO DE LAS VARIABLES

Una función solo puede utilizar las variables de ámbito local, es decir, sus propias variables (los parámetros de la cabecera y las variables creadas dentro de la función). Cuando una función se ejecuta se crean sus variables, se utilizan y cuando la función termina se destruyen las variables.

Por todo ello, una función no puede utilizar variables que estén fuera de ella (las variables globales están prohibidas fuera de la POO), y desde fuera de una función no es posible utilizar variables de la propia función. A esta característica se le llama **encapsulamiento** y permite que las funciones sean independientes entre sí, facilitando el diseño de programas grandes y complejos, buscando una forma de controlar el *acceso a los datos*.

⚡ Técnicamente sí es posible que una función utilice variables que están fuera de ella, pero eso lo veremos cuando aprendamos Programación Orientada a Objetos.

Veamos un ejemplo de los posibles ámbitos habituales:

```
public class Ambitos {  
  
    public static void funcionPruebas(int num){  
        int y = 7;  
        System.out.println(num * 2);  
  
        System.out.println(x * 2);  
  
        System.out.println(y);  
    }  
  
    public static void main(String[] args) {  
  
        int x = 5;  
  
        for(int i = 0; i < 10; i++){  
            System.out.println(i);  
        }  
  
        System.out.println(i);  
        System.out.println(x);  
    }  
}
```

Ámbito local de "funcionPruebas"

Error: x no se ve en este ámbito

Ámbito local de "main"

Ámbito de bloque

Error: i no se ve en este ámbito

4. DISEÑO DE FUNCIONES

A la hora de crear un método o una función, es importante que sigamos unos pasos con el objetivo de garantizar un buen trabajo en su construcción:

1. Describir el objetivo o propósito

Detallar claramente lo que la función debe conseguir. Esto ayudará a acotar el alcance y a asegurar que cumple los objetivos. Por ejemplo, si queremos crear un método para calcular el área de un círculo, debemos describir que el método recibirá como entrada el radio del círculo y devolverá el área como un número decimal.

2. Determinar parámetros de entrada

Qué datos debemos proporcionarle a la función para que pueda realizar su trabajo. Definiremos los tipos de datos de cada uno y el nombre que le daremos. En el ejemplo anterior, el parámetro de entrada es el radio del círculo, y su tipo de datos debería ser decimal (float o double).

3. Determinar el retorno (si lo debe haber) y su valor

Debemos decidir qué información debe devolver la función al finalizar su ejecución, y especificar el tipo de datos del valor de retorno (si lo hay). En el ejemplo anterior, el método devolverá el área del círculo como un número decimal (float o double).

4. Escribir el cuerpo de la función

Desarrollar el conjunto de instrucciones que llevarán a cabo las acciones a cumplir, utilizando los parámetros proporcionados y trabajando con ellos según el propósito a alcanzar. Estas instrucciones son el “algoritmo”. En el ejemplo anterior, sería escribir la fórmula del cálculo del área del círculo.

5. Pruebas

Hay que diseñar distintos casos de prueba, que cubran diferentes escenarios posibles, y comprobar si funciona correctamente. Hay que asegurarse de probar con datos válidos e inválidos para garantizar la robustez. Hay que verificar que la función devuelva los resultados esperados.

Recomendaciones y buenas prácticas en el diseño de funciones:

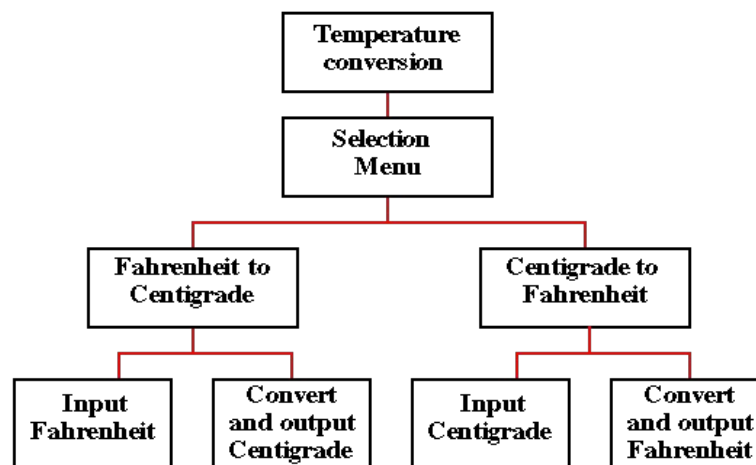
- Nombres significativos: que reflejen claramente su propósito y acción.
- Evitar efectos secundarios: una función debería hacer lo que indica su nombre sin efectos secundarios inesperados.
- Limitar cantidad de parámetros: un exceso de parámetros puede hacer que una función sea difícil de entender y usar. Se debe mantener una cantidad reducida.
- Funciones “pequeñas”: más fáciles de entender y mantener. Si una función es demasiado larga, hay que replantearse el diseño.

5. DISEÑO DESCENDENTE Y MODULARIDAD

El diseño descendente es una técnica que permite diseñar la solución de un problema con base en la modularización o segmentación, dándole un enfoque de arriba hacia abajo. Esto significa partir del problema general e ir separándolo en subproblemas más pequeños, que se pueden tratar de forma separada. Estas separaciones nos permitirán identificar las funciones de nuestra aplicación. Algunas cosas a tener en cuenta al aplicar la metodología:

- Si un problema, que a priori es muy complejo, se descompone en pocos niveles, debemos revisarlo; tal vez estemos incumpliendo la cohesión.
- Si un problema no muy complejo se divide en muchos niveles, probablemente hayamos dividido demasiado y deberíamos pensar en aglutinar.

Veamos un ejemplo de diagrama representando un diseño descendente (top-down), para obtener las partes (o módulos) de una aplicación que representa un “Convertor de temperaturas”:



5.1 Independencia funcional: acoplamiento y cohesión

Se trata de un concepto importante que deriva directamente de la programación modular. Consiste en diseñar los módulos de un programa de forma que realicen una función muy específica y tengan una mínima interdependencia con otros módulos. Esto implica que los módulos deben ser independientes y comunicarse solo mediante la información necesaria para realizar su función.

La independencia funcional es clave para lograr una modularidad efectiva, lo que significa que el software se divide en módulos que se pueden resolver de manera independiente y luego recomponer para obtener la solución al problema original. No existen métodos formales para determinar cómo descomponer un problema en módulos. Es una labor subjetiva que requiere experiencia, sin embargo, existen algunos criterios que pueden guiarnos a la hora de modularizar y se basan en lograr una buena cohesión y acoplamiento. La independencia funcional se mide mediante 2 criterios cualitativos: la **cohesión** y el **acoplamiento**.

- La **cohesión** mide lo específica que es una función, es decir, que realice una tarea concreta, y no se dedique a abarcar varias diferentes. La cohesión ideal debe ser **alta** (alta especificidad).

- El **acoplamiento** es el grado de interdependencia entre los módulos, es decir, mide la relación de un módulo con el resto. El acoplamiento ideal debe ser **bajo**, y puede haber acoplamiento de varios tipos:
 - de **datos** (un módulo accede a datos de otro módulo)
 - de **control** (un módulo controla el comportamiento de otro).

Lo ideal es que el acoplamiento sea sólo funcional, es decir, que un módulo se comuniqué con otro porque le interese la funcionalidad que éste le proporciona.

5.2 Ventajas de un buen diseño descendente

Un **buen diseño descendente** tiene estas ventajas:

1. **Facilita la comprensión y legibilidad del código:** al descomponer un problema en módulos más pequeños y manejables, el código resultante es más fácil de entender. Cada módulo se centra en una tarea específica, lo que mejora la legibilidad y facilita la comprensión del sistema en su conjunto.
2. **Facilita el mantenimiento del código:** los cambios, correcciones o actualizaciones pueden realizarse en módulos individuales sin afectar otras partes del sistema. Esto simplifica la identificación y solución de problemas, reduciendo el riesgo de introducir errores durante las actualizaciones.
3. **Aumenta la reutilización de código:** la modularización permite la creación de módulos independientes y reutilizables, que pueden incorporarse en diferentes partes del sistema o incluso en proyectos futuros, lo que promueve la eficiencia y la consistencia en el desarrollo. Esto también produce un tamaño del código globalmente menor.
4. **Facilita las pruebas:** ya que cada módulo puede evaluarse por separado. Las pruebas individuales de módulos permiten identificar y corregir errores de manera más efectiva antes de integrar todo el sistema. Además, la independencia funcional de los módulos facilita la creación de pruebas automatizadas.
5. **Mejora la escalabilidad:** a medida que los requisitos cambian o se agregan nuevas funcionalidades, es más sencillo integrar nuevos módulos o modificar los existentes sin afectar la estabilidad del sistema en su conjunto.
6. **Facilita el trabajo en equipo en el desarrollo:** la división del problema en módulos más pequeños facilita la asignación de tareas a diferentes desarrolladores o equipos. Cada equipo puede trabajar en un módulo específico de manera independiente y de forma eficiente, lo que agiliza el proceso de desarrollo y permite una colaboración más efectiva.
7. **Fomenta la abstracción y la claridad conceptual:** ya que permite concentrarse en conceptos específicos sin verse abrumado por la complejidad general del sistema. Esto facilita la conceptualización y la planificación del desarrollo.

Estas ventajas son esenciales para lograr sistemas robustos, flexibles y fáciles de mantener a lo largo del tiempo.

6. RECURSIVIDAD

La recursividad es un concepto fundamental en programación modular, y una potente técnica de programación, que implica que una función se llame a sí misma para resolver instancias más pequeñas del mismo problema, evitando el uso de bucles. Esta técnica permite abordar problemas complejos dividiéndolos en subproblemas más simples y manejables.

Características de la Recursividad en Java:

- a) Llamada a sí misma: en una función recursiva, ésta realiza una llamada a sí misma para abordar una versión reducida del problema original.
- b) Caso base: cada función recursiva debe tener un caso base, que devuelve un valor literal, y que indique cuándo dejar de llamarse a sí misma. Este caso base evita la recursividad infinita.

Ejemplo de recursividad en Java

El cálculo del factorial de un número es un problema clásico que se puede resolver mediante recursividad. Veamos su definición y, luego, cómo implementamos esto en Java:

$$0! = 1$$
$$n! = 1 \times 2 \times 3 \times 4 \times \dots \times (n - 1) \times n$$

```
public class Recursividad {  
  
    public static int factorial(int n) {  
        if (n == 0) {  
            return 1; // Caso base  
        } else {  
            return n * factorial(n - 1); // Llamada recursiva  
        }  
    }  
  
    public static void main(String[] args) {  
        int resultado = factorial(5);  
        System.out.println("El factorial de 5 es: " + resultado);  
    }  
}
```

En este ejemplo, la función **factorial** calcula el factorial de un número n . La función se llama a sí misma con un valor reducido ($n - 1$) hasta que alcanza el caso base (cuando n es 0), momento en el cual la recursividad se detiene y se devuelve el resultado.

Ventajas e inconvenientes de la recursividad

Los principales **inconvenientes** del uso de la recursividad radican en su aparente **complejidad** (cuesta ver cómo aplicarla), y en que el abuso de llamadas recursivas puede provocar un **desbordamiento** de la pila de llamadas (en memoria).

Sin embargo, entre las **ventajas** de la recursividad podemos citar que, una vez que se domina, en ocasiones es más sencillo encontrar con ella la solución a un problema. Algunos casos representativos son el algoritmo quicksort de ordenación de arrays, o el recorrido de estructuras en árbol.

7. LIBRERÍAS Y API

Las librerías y APIs son herramientas esenciales para los programadores. Proporcionan una gran cantidad de funcionalidad predefinida que puede ayudar a crear aplicaciones más eficientes y fáciles de mantener.

Una **librería** es una colección de código fuente que proporciona funcionalidad concreta. Las librerías se pueden utilizar para realizar una variedad de tareas, como la manipulación de datos, la entrada y salida de datos, la comunicación de red, el desarrollo de aplicaciones gráficas, etc.

Una **API** (Application Programming Interface) es una interfaz que permite a los programas interactuar con otros programas o sistemas. Las APIs se utilizan para proporcionar acceso a servicios o recursos externos, como bases de datos, sistemas operativos, hardware, etc.

En el contexto de **Java**, la **API de Java**, también conocida como la biblioteca estándar de Java, es un conjunto de librerías que proporciona clases y métodos listos para ser utilizados en el desarrollo de aplicaciones. Estas librerías se organizan en ramas, como "**java.lang**", "**java.util**", "**java.io**", entre otras, y ofrecen funcionalidades para tareas comunes.

Tipos de librerías y APIs

Las librerías y APIs se pueden dividir en dos tipos principales:

- **Librerías y APIs integradas:** Estas librerías y APIs se incluyen con el lenguaje de programación (ya se han usado anteriormente, como por ejemplo para la clase Scanner, la clase Math, etc...)
- **Librerías y APIs de terceros:** Estas librerías y APIs se desarrollan por terceros y no se incluyen con el lenguaje de programación. Son necesarias para hacer cosas muy concretas pero que Java no incluye en su estándar (por ejemplo, para acceder a datos de una red social, conectarse a una aplicación externa, usar una base de datos especial, etc...)